

Lunyard - Code-Erklärung

Was machen die einzelnen Dateien der App?

Die App ist eine Electron-Desktop-App. Vereinfacht: main.js startet das Fenster, index.html ist das Gerüst der Oberfläche, styles.css macht das Aussehen, und renderer.js enthält die gesamte Logik. preload.js verbindet Fenster und System sicher.

index.html

Das HTML-Gerüst der Oberfläche (kein Code-Verhalten, nur Aufbau).

- Lädt die Datei styles.css (Design) und renderer.js (Logik).
- Steuerleiste oben: Anzahl Felder, Backyard-Einstellungen (Startzeit, Rundenzeit, Rundenstrecke) und die Buttons (Werte übernehmen, Backyard An/Aus, Reset, Live Mode, Hellmodus, Strava Stats).
- Der Bereich `<div id="grid">` ist der Platz für die Karten/Felder der Läufer.
- Der untere `<footer id="backyardStrip">` ist der Info-Streifen: Countdown, Status-Pills, Fortschrittsbalken, Uhr und Teilnehmer-Liste.
- Wichtig: Hier stehen nur die IDs/Klassen. Was passiert, steuert renderer.js; wie es aussieht, styles.css.

main.js

Der Hauptprozess von Electron - startet und steuert das Fenster.

- `createWindow()` erstellt das Programmfenster (1400x900), setzt das App-Icon und lädt index.html.
- Aktiviert `webviewTag`: damit können fremde Webseiten (die Strava-Karten) eingebettet werden.
- Bindet `preload.js` ein (sichere Brücke) und schaltet gefährliche Node-Zugriffe im Fenster ab.
- Registriert die Escape-Taste global und schickt das Signal an das Fenster (zum Verlassen des Live-Modus).
- Reagiert auf Nachrichten aus dem Fenster: `set-fullscreen` (Vollbild an/aus) und `set-chrome-bg` (Hintergrundfarbe).
- Schließt die App, wenn alle Fenster zu sind.

package-lock.json

Automatisch von npm erzeugt - die genaue Paketliste.

- Hält exakt fest, welche Pakete in welcher Version installiert wurden (inkl. Unter-Abhängigkeiten).
- Sorgt dafür, dass 'npm install' überall die gleichen Versionen installiert (reproduzierbar).
- Wird nicht von Hand bearbeitet - npm pflegt die Datei selbst.

package.json

Die Steckbrief- und Konfigurationsdatei des Projekts.

- Allgemeine Infos: Name, Version, Beschreibung, Lizenz.
- "main": "main.js" - das ist der Einstiegspunkt, der beim Start ausgeführt wird.
- scripts: kurze Befehle, z. B. "start" (startet die App mit 'electron .') sowie icon/pack/dist.
- devDependencies: benötigte Werkzeuge (electron, electron-builder).
- build: Einstellungen zum Verpacken (App-Name, Icon, Windows-Programm).

preload.js

Sichere Brücke zwischen Fenster (renderer) und System (main).

- Stellt im Fenster ein kleines, sicheres Objekt `window.desktopApi` bereit.
- Funktionen darin: `setFullscreen()` (Vollbild), `setChromeBackground()` (Fensterfarbe), `onGlobalEscape()` (Escape).
- Diese Aufrufe werden per Nachricht an main.js weitergeleitet. So bleibt das Fenster sicher (kein direkter System-Zugriff).

renderer.js

Das Herz der App - die gesamte Logik im Fenster. Die wichtigsten Abschnitte:

- Element-Referenzen & Zustand: holt die HTML-Elemente und merkt sich Variablen (Anzahl Felder, Rundenstrecke, Backyard-Zustand, eine Liste 'participants' fuer alle Laeuer).
- Kleine Helfer: setText() (Text in ein Element schreiben), setAvatar() (Profilbild setzen), escapeHtml(), webViewUrl().
- Formatierung & Parsing: wandelt Texte um - z. B. "5,2 km" in eine Zahl, Sekunden in "m:ss", Pace, Uhrzeit und Countdown.
- Theme / Live Mode / Stats: Hell-/Dunkelmodus, Vollbild-Modus und Strava-Stats ein-/ausblenden.
- Runden-Tracking ohne Backyard: zaehlt Runden ueber Pause/Neustart der Strava-Aktivitaet (Standardfall).
- Backyard-Teil (Kernstueck):
 - getParticipant(): legt pro Laeuer einen Datensatz an (Distanz, Zeiten, ausgeschieden, Rundenzeiten).
 - backyardOnData(): uebernimmt die von Strava ausgelesenen Werte in diesen Datensatz.
 - handleRoundChange(): beim Stundenwechsel - wer das Ziel nicht geschafft hat, scheidet aus; danach starten alle wieder bei 0.
 - renderBackyardPanel(): zeigt pro Laeuer Runde, eine echte Sekundenuhr (laeuft mit, friert beim Ziel ein), Pace und das Diagramm; beim Ausscheiden ein Overlay.
 - renderRoundChart(): kleines Liniendiagramm der Rundenzeiten (Y = Zeit, X = Runde).
 - renderRunners / renderParticipants / renderInfo: bauen den unteren Info-Streifen (Balken mit Profilbildern, Status, Countdown, Uhr).
 - backyardTick(): laeuft jede Sekunde, berechnet die aktuelle Runde und friert den Timer ein, wenn alle ausgeschieden sind.
 - saveBackyard() / toggleBackyard() / resetBackyard(): die Aktionen der drei Buttons.
- Strava-Beacon & Webview: isStravaBeaconUrl() erkennt Strava-Links; buildBeaconScrapeScript() liest Name, Bild, Distanz und Zeit aus der Strava-Seite; ein Poller holt alle 2 Sekunden neue Werte; updateBeaconPanel() schreibt sie in das Panel.
- Raster aufbauen: renderGrid() erzeugt die Felder - leer, als normale Webseite oder als eigenes Strava-Panel.
- Events & Init (am Ende): verbindet die Buttons mit den Funktionen, laedt gespeicherte Einstellungen und baut das Raster auf.

start.bat

Windows-Startdatei zum Doppelklicken.

- Prueft, ob Node.js installiert ist (sonst Hinweis zum Download).
- Installiert bei Bedarf einmalig die Abhaengigkeiten (npm install).
- Startet danach die App (npm start).

styles.css

Das komplette Design (Aussehen) - hier passiert keine Logik.

- Definiert Farben als Variablen (z. B. --beacon-accent = Orange) und einen hellen Modus (theme-light).
- Gestaltet Steuerleiste und Buttons (mit Hover/Fokus, oranger Primaerbutton) sowie das Raster.
- Gestaltet die Strava-Panels: Kopf mit Profilbild/Name, Karte, Runden-Anzeige, Pace, Diagramm und das Ausgeschieden-Overlay.
- Gestaltet den unteren Info-Streifen: Fortschrittsbalken, Laeuer-Punkte, Status-Pills, Chips und Uhr.
- Regelt den Live-Modus (Steuerleiste ausblenden, alles auf Vollbild).